



MET CS 688

Dimensionality Reduction and Data Decomposition II

Leila Ghaedi – Fall 2024

Creator: cemagraphics | Credit: Getty Images/iStockphoto

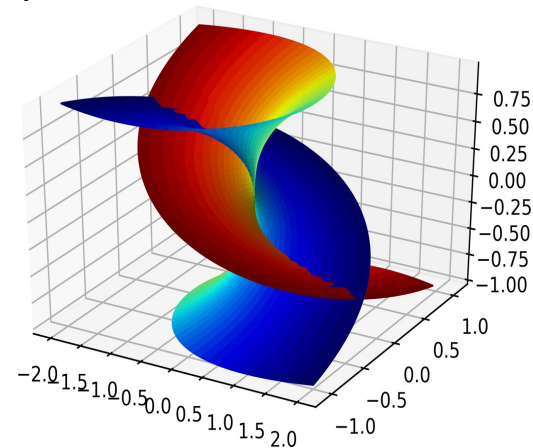


Linear vs Nonlinear Dimensionality Reduction

- Linear:
 - Principal Component Analysis
 - Linear Discriminant Analysis
- Nonlinear:
 - Manifold Dimensionality Reduction
 - t-distributed Stochastic Neighbor Embedding (t-SNE)
 - Uniform Manifold Approximation and Projection for Dimension Reduction (UMAP)

UMAP

- Uniform Manifold Approximation and Projection (UMAP) [McInnes '18] is a dimensionality reduction technique that uses manifold learning.
- It works for datasets that have Riemannian manifold structure. Riemannian manifold are manifolds that planes are smoothly connected to each other, and Riemannian surface planes are connected continuously. In other words, Riemann surface is a multidimensional surface that does not have a 2D plane.



UMAP

- UMAP uses exponential distribution in the high dimensional space, instead of gaussian distribution . Besides, UMAP can work with any distance metrics, but t-SNE is limited to use Euclidean distance only.
- UMAP uses a family of curves in the low dimensional space, which are very similar to t-distribution, but not exactly t-distribution.
- UMAP does not apply any normalization on both high and low dimensional space. This makes it a much faster than tSNE.

Data Decomposition

Data Decomposition

- Data decomposition is a statistical technique that involves breaking down data into smaller components. This can help to better understand the data and make better predictions.

Decomposition of Time Series

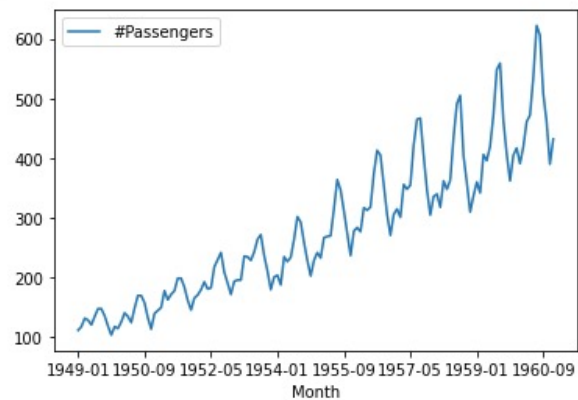
- Time series decomposition is a technique used to break down a time series data into its constituent components.
- The components are:
 1. Trend (long term movement or direction of data)
 2. Seasonality (periodic changes)
 3. Residual (noise)

The decomposition model could be **additive** of **multiplicative**.

Decomposition of Time Series Example

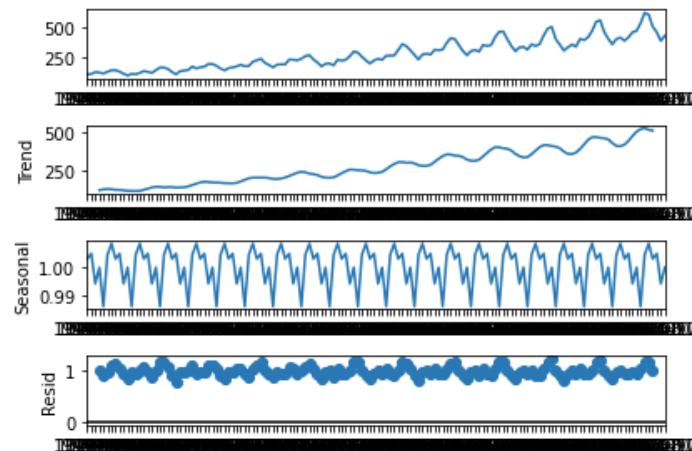
```
In [1]: 1 from random import randrange
        2 from pandas import Series
        3 from matplotlib import pyplot
        4 from statsmodels.tsa.seasonal import seasonal_decompose
        5 from pandas import read_csv
        6 from matplotlib import pyplot
```

```
In [2]: 1
        2 series = read_csv('AirPassengers.csv', header=0, index_col=0)
        3 series.plot()
        4 pyplot.show()
```



Decomposition of Time Series Example

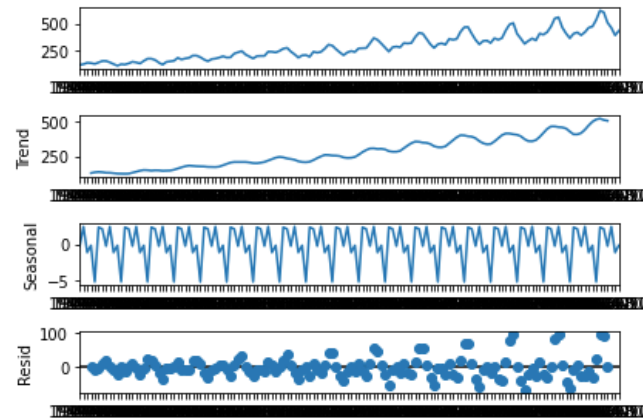
```
In [3]: 1 result = seasonal_decompose(series, model='multiplicative', period=7)
        2 result.plot()
        3
        4 pyplot.show()
```



Decomposition of Time Series Example

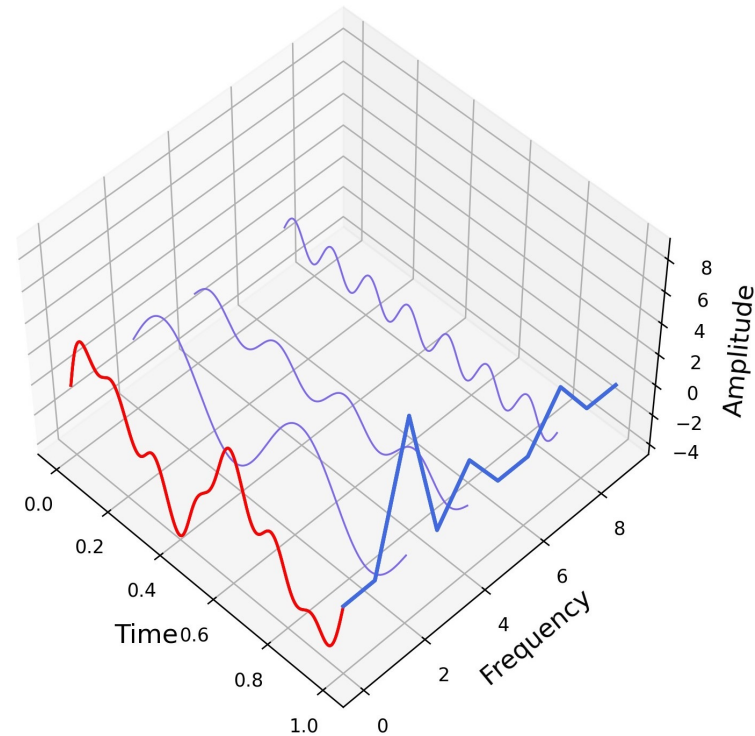
In [4]:

```
1 result = seasonal_decompose(series, model='additive', period=7)  
2 result.plot()  
3  
4 pyplot.show()
```

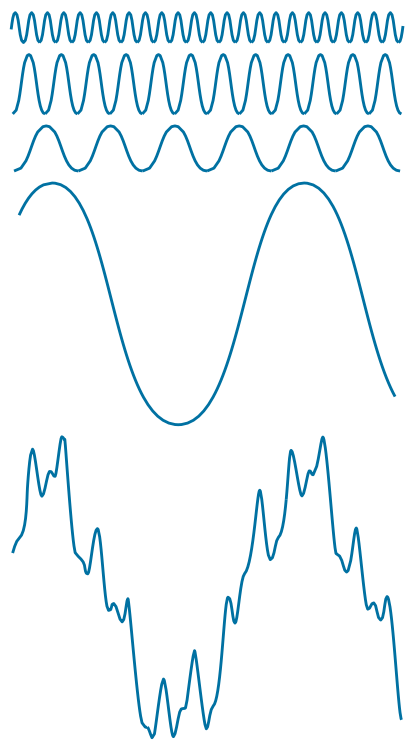


Fourier Series

- Fourier states: “any periodic signal can be represented as a sum of sines, cosines of different frequencies each multiplied by different coefficient.”
- This representations are called Fourier Series and after it has been accepted it has revolutionized several fields including physics and mathematics.



Fourier Series



↑ **Decompose**

The process of transforming a signal into a set of sinusoid called **Decomposition**.

↓ **Synthesis**

The process in the opposite direction, i.e. getting the original signal from sinusoid signals called **Synthesis**.

Signal Types and Transformation

- There are signals that are not having a repetitive patterns (not periodic) those signals are called “aperiodic” signals.
- Also, any signal could be “continuous” or “discrete”. In summary, we could have four types of signals: “aperiodic- continues”, “periodic-continuous”, “aperiodic-discrete” and “periodic-discrete”.

Signal Type	Type of Transformation
Aperiodic- Continuous	Fourier Transform
Periodic- Continuous	Fourier Series
Aperiodic- Discrete	Discrete Time Fourier Transform (DTFT)
Periodic- Discrete	Discrete Time Fourier Transform (DTFT)

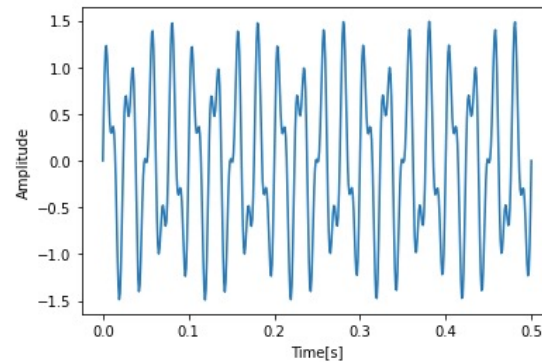
Discrete Fourier Transform

- Discrete Fourier Transform (DFT), is an algorithm that is used to transform periodic-discrete signals into set of sinusoids, and it is the mostly wide used ones in signal processing. The other types of transformation were rarely used in signal processing.
- DFT is getting a time domain signal and transforms (decomposes) it into frequency domain signal.
- Fast Fourier Transform (FFT) is the most popular implementation of a DFT. FFT similar to DFT, gets an array of time domain signal as input, processes it and converts it into frequency domain signals as output

FFT Example Python

```
In [1]: 1 import numpy as np
        2 import matplotlib.pyplot as plt
        3
```

```
In [2]: 1 # create a sample signal
        2 t= np.linspace(0,0.5,500)
        3 s= np.sin(40*2* np.pi*t)+ 0.5* np.sin(90*2* np.pi*t)
        4 # sample signal
        5 plt.ylabel("Amplitude")
        6 plt.xlabel("Time[s]")
        7 plt.plot(t,s)
        8 plt.show()
        9
```



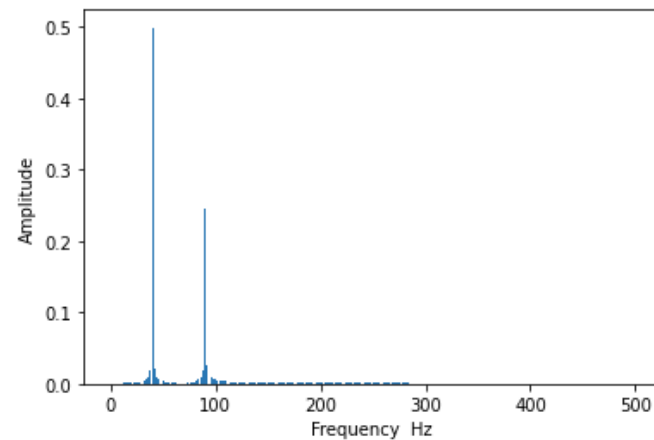
FFT Example Python

```
In [3]: 1 # transform to frequency by FFT
        2 fft=np.fft.fft(s)
        3 print(fft)
```

```
[ 4.44089210e-14+0.00000000e+00j  3.80483493e-04-6.05550318e-02j
 1.53177148e-03-1.21888085e-01j  3.48410350e-03-1.84815540e-01j
 6.29040694e-03-2.50234639e-01j  1.00304815e-02-3.19175096e-01j
 1.48177406e-02-3.92866554e-01j  2.08096627e-02-4.72832115e-01j
 2.82239361e-02-5.61024391e-01j  3.73637686e-02-6.60032056e-01j
 4.86586362e-02-7.73406876e-01j  6.27323240e-02-9.06205150e-01j]
```

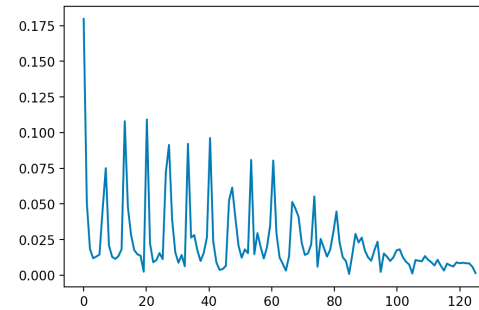
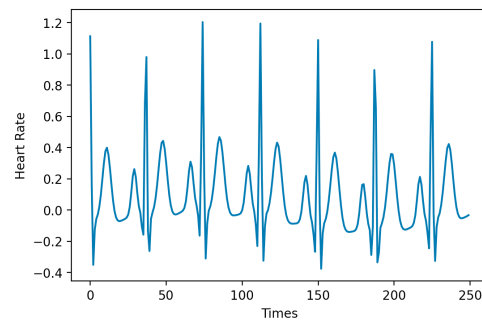

FFT Example Python

```
In [4]: 1 T=t[1]-t[0]
        2 N=s.size
        3 #1/T=frequency
        4 f=np.linspace(0,1/T,N)
        5 plt.ylabel("Amplitude")
        6 plt.xlabel("Frequency Hz")
        7 plt.bar(f[: N//2], np.abs(fft)[: N//2]/N, width=1.5)
        8 plt.show()
        9
```



Challenges of Fourier Transformation

We lose the notion of time when a signal is transferred into a frequency-domain signal. For example, Electrocardiography (ECG) signals have short intervals of oscillation, and this characteristic can not be captured by Fourier transforms.



Wavelet Transform

In the data science community, wavelet transformation receives more attention than Fourier series, because they provide a better resolution to the original signal.

Wavelet are mini waves, which unlike waves does not go forever. Waves such as sine or cosine waves are going forever, but wavelet has a finite start and stop time.

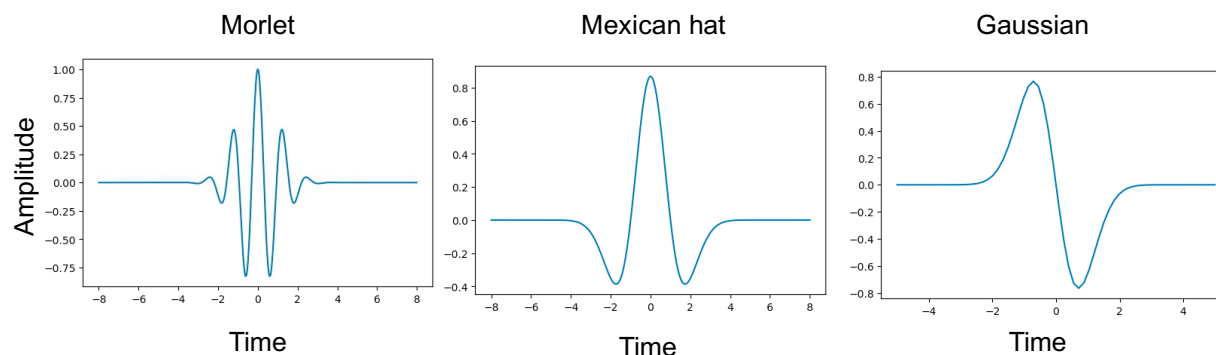
Wavelets have finite time and frequency.

When we apply wavelet transform, we do not get data in frequency domain anymore. The output is in wavelet domain.

A wavelet get squeezed or stretched, i.e. scaled, on x axis to produce all output signals (wavelet signals). Therefore, all wavelet transformation algorithms are associated with a “scaling” function.

Continuous Wavelet Transform

Some examples of three common Continuous Wavelet Transform (CWT) wavelet shapes. Unlike a sinusoid wave that goes to infinity, we can see that the wavelet is a decaying wave (right and left side value is going toward zero) that has zero mean.



Wavelet is localized in time; we can multiply our signal with the wavelet at different locations in time. We start with the beginning of our signal and slowly move the wavelet towards the end of the signal. This procedure is also known as a convolution. After we have done this for the original (mother) wavelet, we can scale it such that it becomes larger and repeat the process.

source: <https://ataspinar.com/2018/12/21/a-guide-for-using-the-wavelet-transform-in-machine-learning>

Discrete Wavelet Transform

- Discrete Wavelet Transform (DWT) [Heil '89] is widely used wavelet transformation.
- FFT only converts the given signal into frequency domain signals, FFT only output frequency domain signals. However, DWT (similar to other wavelet transforms) maintains information about time domain signal as well.
- While the FFT is a powerful tool for analyzing the frequency content of a signal, the DWT is preferred in cases where both frequency and temporal information are crucial, especially for signals with time-varying characteristics.

Discrete Wavelet Transform Example

```
In [1]: 1 import pywt
        2 (cA, cD) = pywt.dwt([1,2,3,4,5,6], 'haar')
        3 #(cA, cD)tuple
        4 # Approximation and detail coefficients.
```

```
In [2]: 1 print(cA)

[2.12132034  4.94974747  7.77817459]
```

```
In [3]: 1 print(cD)

[-0.70710678 -0.70710678 -0.70710678]
```

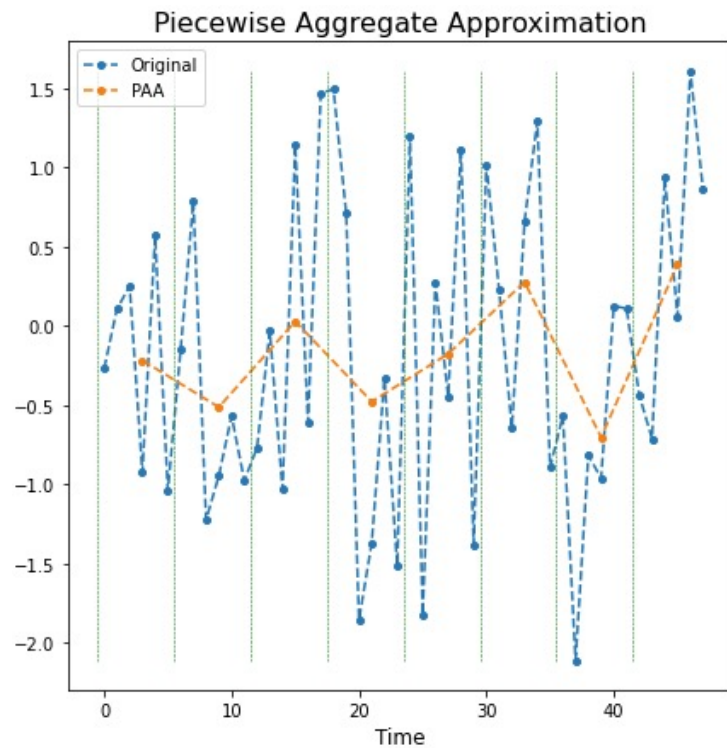
```
In [4]: 1 # reconstruct the original signal
        2 # using inverse discrete wavelet transform
        3 print(pywt.idwt(cA,cD,'db1'))

[1.  2.  3.  4.  5.  6.]
```

Approximate Aggregation Methods

- Piecewise Aggregate Approximation (PAA)
- Symbolic Aggregate Approximation (SAX)

Piecewise Aggregate Approximation (PAA)



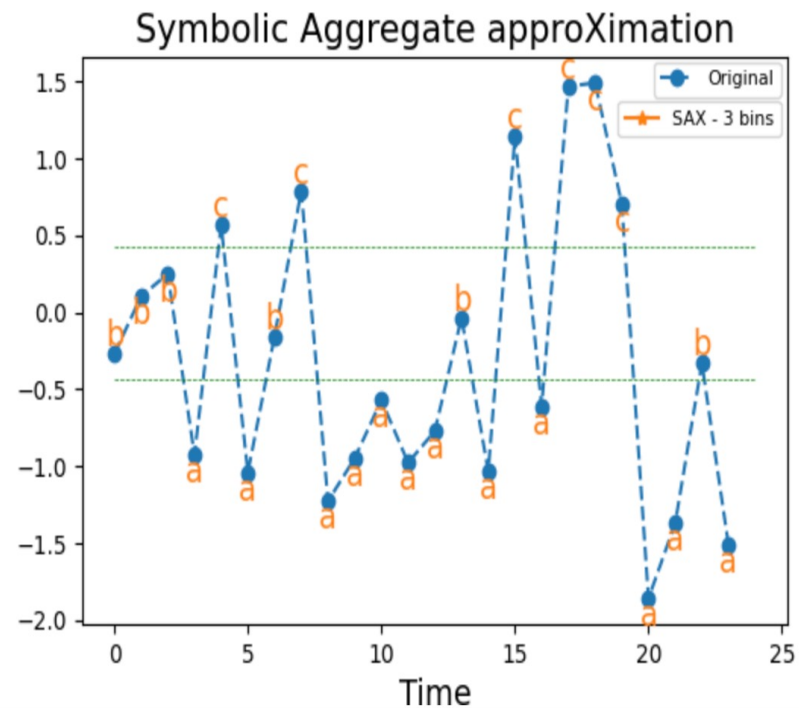
PAA is reducing the time series size, by splitting it into equal size segments. Then, for each segment an average value of the datapoints will be calculated and substituted the original data. Therefore, the time series will be transformed into a simpler model, which holds most of the characteristics from the original one as well.

1. Segmentation: The time series is divided into equal-sized, non-overlapping segments.
2. Averaging: For each segment, the average value of the data points within that segment is calculated.
3. Representation: The original time series is then represented by the sequence of average values obtained from each segment.

Symbolic Aggregate Approximation (SAX)

It is particularly useful for reducing the dimensionality of time series data while preserving its essential characteristics.

Binning continuous data into intervals can be seen as an approximation that reduces noise and captures the trend of a time series. The SAX algorithm bins continuous time series into intervals, transforming independently each time series (a sequence of floats) into a sequence of symbols, usually letters. This example illustrates the transformation.



Symbolic Aggregate Approximation (SAX)

1. Normalization: The time series is first normalized to have zero mean and unit standard deviation to make it comparable across different scales.
2. Alphabet Creation: SAX defines an alphabet of symbolic letters. The size of the alphabet is a user-defined parameter that determines the granularity of the symbolic representation.
3. Symbolization: The normalized time series is divided into non-overlapping segments. For each segment, a representative letter from the alphabet is assigned based on the probability distribution of values within that segment.
4. Representation: The original time series is then represented by the sequence of symbols obtained from each segment.

https://pyts.readthedocs.io/en/latest/auto_examples/approximation/plot_sax.html

Matrix Decomposition in Machine Learning (Matrix Factorization)

- Matrix decompositions are methods that break down a matrix into component parts.
- Factoring is the process of making an object combination of smaller object, e.g. 15 will be factorized into 3 x 5. In other words, matrix factorization reduces the size of the matrix, which leads to resource efficiency.
- Matrix factorization is widely used for machine learning, especially for algorithms that require operating with large matrixes and avoid handing over the process to another hardware such as GPU (Graphical processing unit), or NPU (Neural Processing Unit).

Matrix Decomposition Methods

1. LU Decomposition
2. QR Decomposition
3. Singular Value Decomposition (SVD)
4. Cholesky Decomposition
5. Non-Negative Matrix Factorization (NMF)
6. Principal Component Analysis (PCA)

LU Decomposition

- Any non-singular matrix A can be factored into a lower triangular matrix L , and upper triangular matrix U .
- A non-singular matrix is a square matrix with a non-zero determinant.
- Determinant of $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$:
- $|A| = ad - bc$

LU Decomposition

- Any non-singular $n \times n$ matrix A , it can be written as a product
- $A=LU$
- Where L is a lower triangular matrix and U is an upper triangular matrix.

$$\bullet A = \begin{bmatrix} 4 & -2 & -3 & 1 \\ 1 & 3 & 1 & 3 \\ 1 & 2 & -1 & -2 \\ 2 & 1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.25 & 1 & 0 & 0 \\ 0.25 & 0.714 & 1 & 0 \\ 0.5 & 0.571 & 0.333 & 1 \end{bmatrix} \begin{bmatrix} 4 & -2 & -3 & 1 \\ 0 & 3.5 & 1.75 & 2.75 \\ 0 & 0 & -1.5 & -4.214 \\ 0 & 0 & 0 & -1.667 \end{bmatrix}$$

LU Decomposition, Why?

- Let's say we are trying to solve this ($Ax=b$)
- Gaussian Elimination $O(n^3)$

$$\bullet \begin{bmatrix} 4 & -2 & -3 & 1 \\ 1 & 3 & 1 & 3 \\ 1 & 2 & -1 & -2 \\ 2 & 1 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 20 \\ 14 \\ 3 \\ 9 \end{bmatrix}$$

LU Decomposition, Why?

- Let's substitute A with LU:

$$\bullet \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.25 & 1 & 0 & 0 \\ 0.25 & 0.714 & 1 & 0 \\ 0.5 & 0.571 & 0.333 & 1 \end{bmatrix} \begin{bmatrix} 4 & -2 & -3 & 1 \\ 0 & 3.5 & 1.75 & 2.75 \\ 0 & 0 & -1.5 & -4.214 \\ 0 & 0 & 0 & -1.667 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 20 \\ 14 \\ 3 \\ 9 \end{bmatrix}$$

$$\bullet \begin{bmatrix} x_{t1} \\ x_{t2} \\ x_{t3} \\ x_{t4} \end{bmatrix} = \begin{bmatrix} 4 & -2 & -3 & 1 \\ 0 & 3.5 & 1.75 & 2.75 \\ 0 & 0 & -1.5 & -4.214 \\ 0 & 0 & 0 & -1.667 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

- Here we can solve for x_t , by forward substitution, starting from x_{t1} :

$$\bullet \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.25 & 1 & 0 & 0 \\ 0.25 & 0.714 & 1 & 0 \\ 0.5 & 0.571 & 0.333 & 1 \end{bmatrix} \begin{bmatrix} x_{t1} \\ x_{t2} \\ x_{t3} \\ x_{t4} \end{bmatrix} = \begin{bmatrix} 20 \\ 14 \\ 3 \\ 9 \end{bmatrix} \quad x_{t1} = 20, 0.25*20+x_{t2}=14, x_{t2}=9$$

LU Decomposition, Why?

- Now that we solved x_t , we can solve for by backward substitution here:

$$\bullet \begin{bmatrix} 20 \\ 9 \\ -8.42 \\ -3.4 \end{bmatrix} = \begin{bmatrix} 4 & -2 & -3 & 1 \\ 0 & 3.5 & 1.75 & 2.75 \\ 0 & 0 & -1.5 & -4.214 \\ 0 & 0 & 0 & -1.667 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

- $x_4 = 2.00$

$$\bullet x = \begin{bmatrix} 5.0 \\ 1 \\ 0 \\ 2.0 \end{bmatrix}$$

LU Decomposition Python

```
In [1]: 1 import numpy as np
        2 from scipy.linalg import lu, lu_factor, lu_solve
```

```
In [2]: 1 A = np.array([[4,-2,-3,1], [1,3,1,3],
        2                  [1,2,-1,-2] , [2,1,-1,-1]])
        3 P, L, U = lu(A)
        4 # A = P L U
        5 # P is the permutation matrix
        6 print(L)
        7 print(U)
```

```
[[1.         0.         0.         0.        ]
 [0.25       1.         0.         0.        ]
 [0.25       0.71428571 1.         0.        ]
 [0.5        0.57142857 0.33333333 1.        ]]
[[ 4.         -2.         -3.          1.        ]
 [ 0.          3.5         1.75         2.75       ]
 [ 0.          0.         -1.5        -4.21428571]
 [ 0.          0.          0.         -1.66666667]]
```

LU Decomposition Python

```
In [3]: 1 L@U
```

```
Out[3]: array([[ 4., -2., -3.,  1.],  
               [ 1.,  3.,  1.,  3.],  
               [ 1.,  2., -1., -2.],  
               [ 2.,  1., -1., -1.]])
```

```
In [4]: 1 np.allclose(L@U, A)
```

```
Out[4]: True
```

LU Decomposition Python

```
In [5]: 1 # Solve Ax=b
        2 # solve Gaussian Elimination using LU Decomposition
        3 b=np.array([20,14,3,9])
```

```
In [6]: 1 # Pivot indices representing the permutation matrix P
        2 LU, piv=lu_factor(A)
        3 lu_solve((LU,piv),b)
        4 # this is the x
```

```
Out[6]: array([ 5.,  1., -0.,  2.])
```

```
In [7]: 1 print(piv)

[0 1 2 3]
```

```
In [8]: 1 # in this case permutation matrix is identical matrix
        2 print(P)

[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

LU Decomposition Python

```
In [9]: 1 # solving LU and Gaussian Elimination is  $O(n^3)$ 
2 # in case the b vector keeps changing and A is constant
3 # we save time by doing LU decomposition once and solve
4 #  $Ax=b$  with forward and backward equations  $O(n)$ 
5
6 np.random.seed(10)
7 N=100
8 A=np.random.normal(size=(N,N))
9 b=np.random.normal(size=(N,))
```

```
In [10]: 1 start = time()
2 for i in range(1000):
3     x1=np.linalg.solve(A,b)
4     print("Time without LU Decomposition=" ,time()-start)
```

Time without LU Decomposition= 0.4556610584259033

```
In [11]: 1 start = time()
2 lu, piv= lu_factor(A)
3 for i in range(1000):
4     x2=lu_solve((lu,piv),b)
5     print("Time with LU Decomposition=" ,time()-start)
```

Time with LU Decomposition= 0.014016866683959961

QR Decomposition

- QR decomposition (or QR factorization) is a matrix decomposition method. Matrix A represent by QR method as the product of an orthogonal matrix Q and an upper triangular matrix R.
- $A=QR$
- Q: is an orthogonal matrix. $Q^T Q=I$ where I is the identity matrix.
- R: is an upper triangular matrix.

QR Decomposition Example

```
In [1]: 1 import numpy as np
        2 from scipy.linalg import qr
        3
        4 A = np.random.randn(9, 3)
        5 Q, R = np.linalg.qr(A, mode='reduced')
        6
        7 A.shape, Q.shape, R.shape
```

```
Out[1]: ((9, 3), (9, 3), (3, 3))
```

```
In [2]: 1 print("Original Matrix A:")
        2 print(A)
```

```
Original Matrix A:
[[ 1.24218806  2.15804111  0.62798206]
 [ 0.25225797 -1.12200919  0.5529848 ]
 [-0.38147646  0.16963623  0.88340269]
 [-0.13643258  0.01898134 -0.59625007]
 [-0.70712616 -0.6760541  1.26733578]
 [ 0.75208022 -0.72779075 -0.83529805]
 [-0.04675289 -1.75967108  0.69663324]
 [-1.67676911 -0.41877429  3.4921653 ]
 [-1.55654514  0.29322052  0.12954289]]
```

QR Decomposition Example

```
In [3]: 1 print("\nOrthogonal Matrix Q:")
        2 print(Q)
```

```
Orthogonal Matrix Q:
[[-0.43718887  0.57213406 -0.5038565 ]
 [-0.08878235 -0.3911134  -0.24166185]
 [ 0.13426088  0.09493397 -0.16658802]
 [ 0.04801753  0.02039813  0.21708946]
 [ 0.2488735   -0.14607583 -0.20286621]
 [-0.26469511 -0.31505669  0.05070337]
 [ 0.01645471 -0.56726279 -0.21065206]
 [ 0.59013995  0.03868576 -0.62117671]
 [ 0.54782705  0.25764675  0.3726774  ]]
```

```
In [4]: 1 # QT@Q= I
        2 print(np.dot (Q.T , Q))
```

```
[[ 1.00000000e+00  1.38777878e-16 -1.11022302e-16]
 [ 1.38777878e-16  1.00000000e+00  8.32667268e-17]
 [-1.11022302e-16  8.32667268e-17  1.00000000e+00]]
```


QR Decomposition Example

```
In [5]: 1 print("\nUpper Triangular Matrix R:")
        2 print(R)
        3 print(Q)
```

```
Upper Triangular Matrix R:
[[-2.84130757 -0.91123548  2.44613576]
 [ 0.          3.07560623  0.06605108]
 [ 0.          0.         -3.29382539]]
[[-0.43718887  0.57213406 -0.5038565 ]
 [-0.08878235 -0.3911134  -0.24166185]
 [ 0.13426088  0.09493397 -0.16658802]
 [ 0.04801753  0.02039813  0.21708946]
 [ 0.2488735  -0.14607583 -0.20286621]
 [-0.26469511 -0.31505669  0.05070337]
 [ 0.01645471 -0.56726279 -0.21065206]
 [ 0.59013995  0.03868576 -0.62117671]
 [ 0.54782705  0.25764675  0.3726774 ]]
```

QR Decomposition Example

In [6]:

```
1 # reconstruct A
2 print(Q@R)
```

```
[[ 1.24218806  2.15804111  0.62798206]
 [ 0.25225797 -1.12200919  0.5529848 ]
 [-0.38147646  0.16963623  0.88340269]
 [-0.13643258  0.01898134 -0.59625007]
 [-0.70712616 -0.6760541   1.26733578]
 [ 0.75208022 -0.72779075 -0.83529805]
 [-0.04675289 -1.75967108  0.69663324]
 [-1.67676911 -0.41877429  3.4921653 ]
 [-1.55654514  0.29322052  0.12954289]]
```

In [7]:

```
1 np.allclose(A, np.dot(Q, R)) # A does equal QR
2
```

Out[7]: True

Cholesky Decomposition

- One way to decompose a number is to take its square root, $16 = 4$.
- For symmetric matrices that include only non-negative numbers, we can also use something similar to square root to decompose them. Cholesky decomposition [Cholesky '10] method proves Matrix A could be decomposed as $A=L.LT$.
- L is a lower triangular matrix (lower triangular matrix means the lower side of the diagonal is all zero) and LT is a transposed of L which will be an upper triangular matrix. Despite its limitation (only works for symmetric matrices) it is supported by most machine learning software to facilitate the overhead of numerical computation.

$$\bullet A = \begin{bmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{00} & 0 & 0 \\ L_{10} & L_{11} & 0 \\ L_{20} & L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} L_{00} & L_{01} & L_{02} \\ 0 & L_{11} & L_{12} \\ 0 & 0 & L_{22} \end{bmatrix}$$

- Time complexity of Cholesky decomposition is $O(n^3)$

Cholesky Decomposition Python

```
In [1]: 1 import numpy as np
        2 import scipy.linalg as la
```

```
In [2]: 1 A=np.array([[1,2,5],[2,13,12],[5,12,42]])
        2 L=la.cholesky(A, lower=True)
        3
        4 print('Cholesky result:')
        5 print(L)
```

```
Cholesky result:
[[1.      0.      0.      ]
 [2.      3.      0.      ]
 [5.      0.66666667 4.06885187]]
```

```
In [3]: 1 print(A)
        2 print(L@L.T)
```

```
[[ 1  2  5]
 [ 2 13 12]
 [ 5 12 42]]
[[ 1.  2.  5.]
 [ 2. 13. 12.]
 [ 5. 12. 42.]]
```

Non-negative Matrix Factorization(NMF)

- Non-negative Matrix Factorization(NMF) is a method used to factorize a non-negative matrix, X , into the product of two lower rank matrices, H and W , such that $H.W$ approximates an optimal solution of X .
- NMF is an unsupervised learning algorithm used to reduce the dimensionality of data into lower-dimensional spaces.
- $X_{n \times m} \approx H_{n \times k} \cdot W_{k \times m}$
- X , H and W are all non-negative.
- Its equation is written as approximation, because it does not guarantee whether we can recover the original matrix from the decomposed ones.

NMF Python

```
In [1]: 1 # NMF Decomposition
        2 import numpy as np
        3 from sklearn.decomposition import NMF
```

```
In [2]: 1 X=np.array([[1,1,3,3,4,7,1], [2,1,1,6,2,6,3],
        2               [3,3,5,1,7,2,1], [1,2,1,7,4,6,1]])
        3 # Implement NonNegative Matrix Factorization
        4 model= NMF(n_components=2,init='random', random_state=0)
        5 W=model.fit_transform(X)
        6 H=model.components_
```

NMF Python

```
In [3]: 1 print('Original')
        2 print(np.shape(X))
        3 print(X)
```

```
Original
(4, 7)
[[1 1 3 3 4 7 1]
 [2 1 1 6 2 6 3]
 [3 3 5 1 7 2 1]
 [1 2 1 7 4 6 1]]
```

```
In [4]: 1 print('W:')
        2 print(np.shape(W))
        3 print(W)
```

```
W:
(4, 2)
[[1.39189231 1.25534113]
 [1.90578274 0.        ]
 [0.19871545 3.43231923]
 [1.95312911 0.44975024]]
```

```
In [5]: 1 print('H:')
        2 print(np.shape(H))
        3 print(H)
```

```
H:
(2, 7)
[[0.55993439 0.55270254 0.44252662 3.12843037 1.28328627 3.34443642
 0.92933994]
 [0.74857535 0.78350953 1.46866529 0.        1.96193179 0.53749542
 0.15217698]]
```

Singular Value Decomposition (SVD)

- SVD is the most used matrix decomposition method that is used to convert (factorize) a matrix into singular vectors and singular values.
- $A = U \Sigma V^T$
- $A_{n \times m}$
- $U_{m \times m}$ is a unitary matrix ($U^T * U = I$)
- $\Sigma_{m \times n}$ is a rectangular diagonal matrix
- $V_{n \times n}$ is a unitary matrix ($V^T * V = I$)
- Columns of U are called left singular vectors of A , and columns of V are called right singular vectors of A . To reduce the dimensionality of the data, we can select top k largest singular values in Σ .

SVD Python Example

```
In [1]: 1 # Singular_Value_Decomposition
        2 import numpy as np
        3 from scipy.linalg import svd
        4
        5 A= np.random.randn(9,4) + 1j*np.random.randn(9,4)
        6 A

Out[1]: array([[ 1.871934 +0.69933761j,  0.10638256+0.48276567j,
                -1.11184129+0.39149418j,  0.40568475-1.46087527j],
               [-1.37573777+0.11356902j, -1.13084679-1.81261328j,
                0.54370719-0.87287954j, -2.5399696 +0.97138712j],
               [ 0.79982542+0.95040295j, -0.83223793+0.08517425j,
                -1.95189102+0.34924819j,  1.83986968-1.65900539j],
               ...])
```

SVD Python Example

```
In [2]: 1 U,S, VT=svd(A)
```

```
In [3]: 1 A.shape, U.shape, S.shape, VT.shape
```

```
Out[3]: ((9, 4), (9, 9), (4,), (4, 4))
```

```
In [4]: 1 U
```

```
Out[4]: array([[ -0.35115617-1.65853426e-01j, -0.05935806+1.51289401e-01j,  
                0.1727484 +8.30288068e-02j, -0.05414052-2.25331767e-01j,  
                0.0223402 -5.41605809e-01j, 0.14452782-3.35795841e-01j,  
                -0.0349076 -9.20827007e-02j, -0.46227266-2.22576446e-01j,  
                -0.09311905+1.52448187e-01j],  
               [ 0.35395183+1.38085829e-01j, 0.47022537-3.12345878e-01j,  
                0.50075641+1.62643015e-02j, -0.13240541-1.88919330e-02j,  
                -0.016139 -1.77474254e-01j, 0.27172745+2.93482715e-01j,  
                -0.03246528-1.81999010e-01j, -0.09417946-7.58710623e-02j,  
                0.1118167 +1.22624067e-01j],
```

SVD Python Example

```
In [5]: 1 S
```

```
Out[5]: array([6.70836748, 4.16454978, 3.34570208, 2.23952942])
```

```
In [6]: 1 VT
```

```
Out[6]: array([[ -0.59765623+0.j, 0.00156533-0.39511533j,
                0.14134472-0.46471049j, 0.05063804+0.49818679j],
               [ -0.47885804+0.j, -0.32599556-0.33400692j,
                -0.25943939+0.31791415j, -0.45430184-0.42201052j],
               [ 0.51820945+0.j, -0.27526301-0.25240575j,
                -0.08567082-0.59642922j, -0.47448501-0.06145929j],
               [ -0.38073741+0.j, 0.03289976+0.69676891j,
                -0.01217727-0.48215354j, -0.1539136 -0.33490297j]])
```

```
In [7]: 1 # Reconstruction
        2 np.allclose(A, np.dot(U[:, :4] * S, VT))
```

```
Out[7]: True
```

SVD vs PCA

- Singular Value Decomposition (SVD) and Principal Component Analysis (PCA) are related concepts but are used in slightly different contexts.
- Purpose:
 - SVD: SVD is a matrix factorization method used to decompose a matrix into three other matrices. It is a general technique that is not specifically designed for dimensionality reduction but is widely used in various applications, including data compression and solving linear systems.
 - PCA: PCA is a specific application of SVD. It is primarily used for dimensionality reduction and feature extraction. PCA identifies the principal components of a dataset, which are linear combinations of the original features that capture the most significant variance.

SVD vs PCA

- Dimensionality Reduction

- SVD: While SVD can be used for dimensionality reduction, it doesn't inherently provide a clear way to select a reduced set of dimensions or components.
- PCA: PCA explicitly aims to find the principal components that retain most of the variance in the data, allowing for effective dimensionality reduction by selecting a subset of these components.

SVD vs PCA

- Data Interpretation
 - SVD: SVD doesn't provide direct insights into the interpretation of the components or their significance in the context of the original features.
 - PCA: PCA components are ordered by the amount of variance they explain, making it easier to interpret and select the most informative features.

SVD is a more general method, where PCA is a specific application of SVD tailored for dimensionality reduction and feature extraction in the context of data analysis.

MET CS 688

Graph Mining Algorithms

Leila Ghaedi – Fall 2024

Creator: cemagraphics | Credit: Getty Images/iStockphoto



Graph Mining Algorithms

Outline

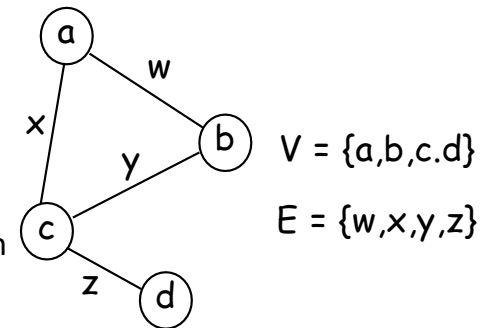
- **Basic Graph Concepts**
- Minimum Spanning Tree
- Shortest Path Finding
- Centrality Measurements
- Link Analysis
- Community Detection

Why Graph Mining?

- Before the revolution of deep learning social media analysis was the hype and have the attention of the machine learning community. Several interesting phenomena, including propagation of influenza, people's attitude toward a specific technology, political opinion analysis are identified by social media.
- Besides, graph algorithms have application in autonomous driving, navigation, network routing, message passing, etc.

Basic Graph Concepts

- A graph is presented as $G = (V, E)$ with edges E and **vertexes** or **nodes**, V . **Edges** or **links** are connecting the vertexes or nodes of graphs together and the relation between.
- If edges has direction, the graph is called directed graph.
- Nodes that are connected with one edge, are called adjacent nodes.
- The process of crossing an edge to get from one node to to another node is called **hop**. For example, in the figure to get from a to d , the graph can make two hops from a to c and then from c to d , or it can get form a to b , b to c and c to d , which results in three hops.
- A **clique** is a subset vertices in a graph that every vertices are adjacent. For example, $\{a, b, c\}$ construct a clique, because they are all adjacent. It does not include d , because there is no direct connection from d to a and b .

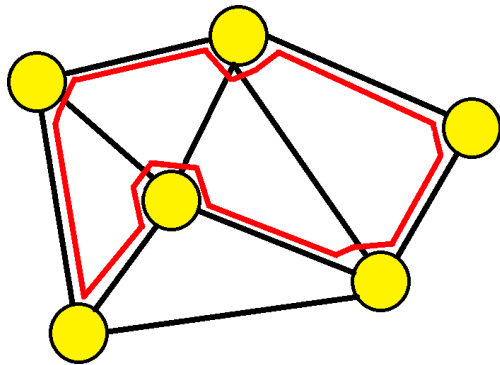


Basic Graph Concepts

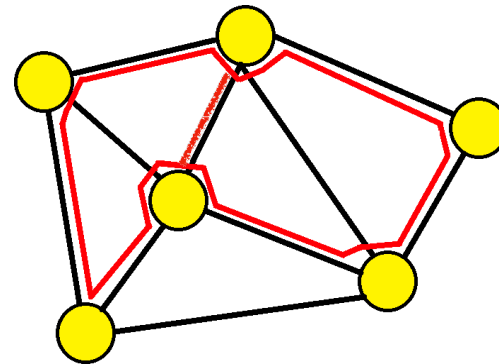
- A shortest path from one node to another node in the network is called **geodesic path**.
- When we travel from a node to another node in a graph, the set of visited nodes and edges are referred as **walk**.
- If the walk get backs to the beginning node it is called **closed walk**, e.g. $\{a, b, c, b, c, a\}$.
- If in a closed walk every node visited exactly once and no more than once, it is called a **Hamiltonian path**, e.g. $\{a, b, c, a\}$.

Hamiltonian Cycle vs None-Hamiltonian Cycle

A graph cycle is a cycle that visits each node exactly once, is referred to as the Hamilton cycle.



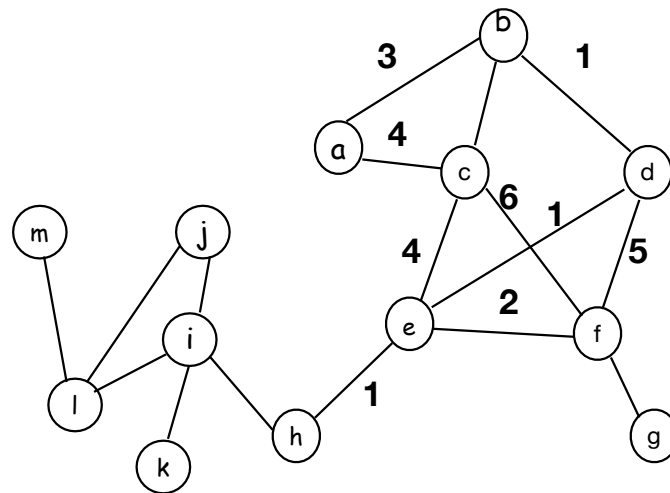
Hamiltonian Cycle



None-Hamiltonian Cycle

Some Examples

- Assign random weight to edges and find a geodesic path from **a** to **h**.
- Find an example for a non-closed walk.
- Find an example for a cycle.
- Find an example for a Hamiltonian cycle.

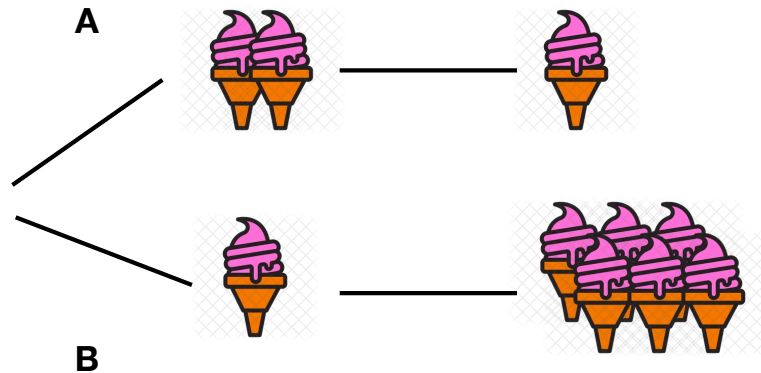


Outline

- Basic Graph Concepts
- **Minimum Spanning Tree (Prim, Kruskal)**
- Shortest Path Finding
- Centrality Measurements
- Link Analysis
- Community Detection

Greedy Algorithm

- Greedy algorithm is a short-sighted algorithm that does not have a long vision, they favor local optima over global optima.

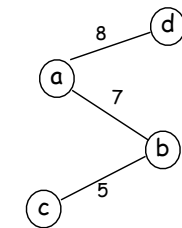
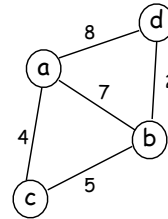


Spanning Tree

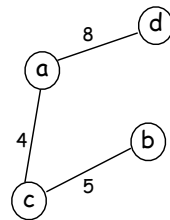
- Spanning trees are graphs that do not have a cycle and removing one more edge from a spanning tree result in having two separate trees (tree is a graph that has no cycle).
- A spanning tree is a connected graph that includes all vertices of a graph with a minimum possible number of edges. A spanning tree does not have cycles and cannot be disconnected.

Minimum Spanning Tree

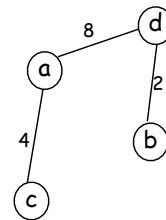
- Finding the minimum spanning tree (MST) has many applications including power distribution design, robot movements in the field, recommendation systems (nodes are product and the similar product stay in the neighbor nodes), etc.
- Usually, MST algorithms are used for cost minimization and because of that they are very popular algorithm.



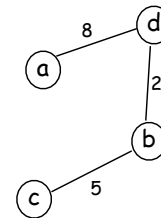
sum of weight=20



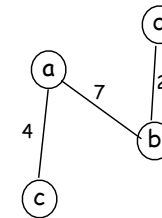
sum of weight=17



sum of weight=14



sum of weight=15



sum of weight=13

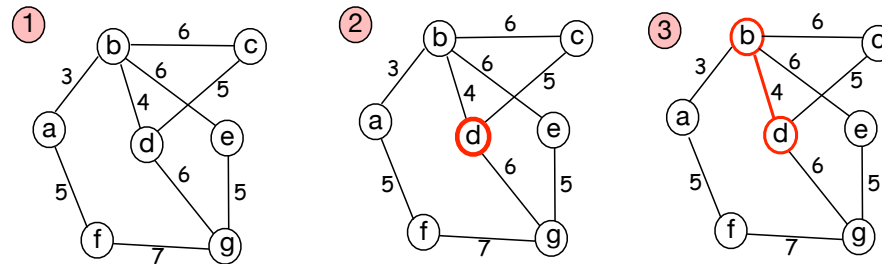
Prim Algorithm

- Prim's algorithm (Jarník '30, Prim '57) is one of the first algorithm for MST identification.
- It uses a greedy method to find MST in weighted and undirected graphs.
- Mazes are drawn by the Prim algorithm as well:
[https:// mtimmerm.github.io/webStuff/maze.html](https://mtimmerm.github.io/webStuff/maze.html)

Prim Algorithm

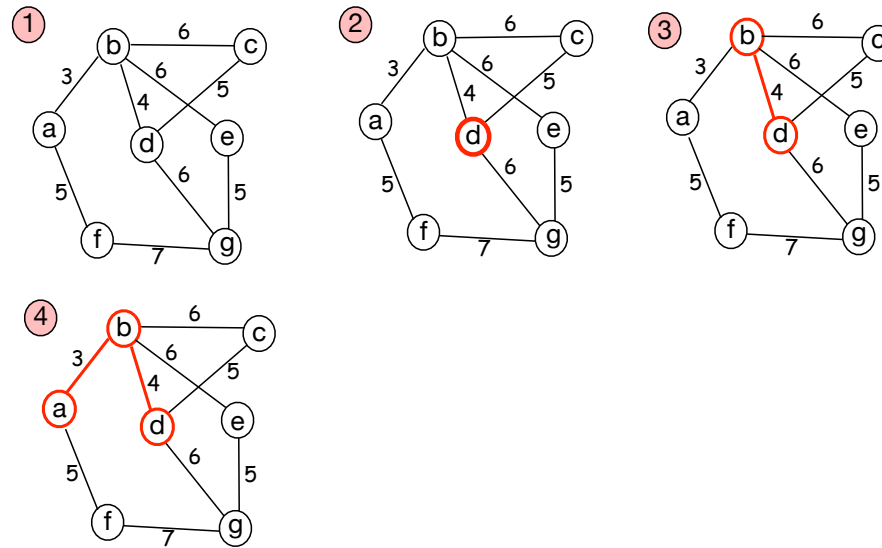
- First, it starts by selecting a random node on the graph. Next, it starts to check all edges from the selected node and grows a tree by connecting to the node which, (i) has the lowest weight, (ii) the connection does not form a cycle. It means the algorithm checks all neighbor nodes of the selected node and grows the tree by one edge. As a result, the randomly selected node of the previous step is connected to the node, which has the lowest cost.
- Next, it again compute the shortest distances from all nodes that have been selected, not just the most recent node, and choose the edge that has shortest distance and does not construct a cycle to expand the tree. This process continues until all edges have been selected and the result is minimum spanning tree.

Prim Algorithm Example

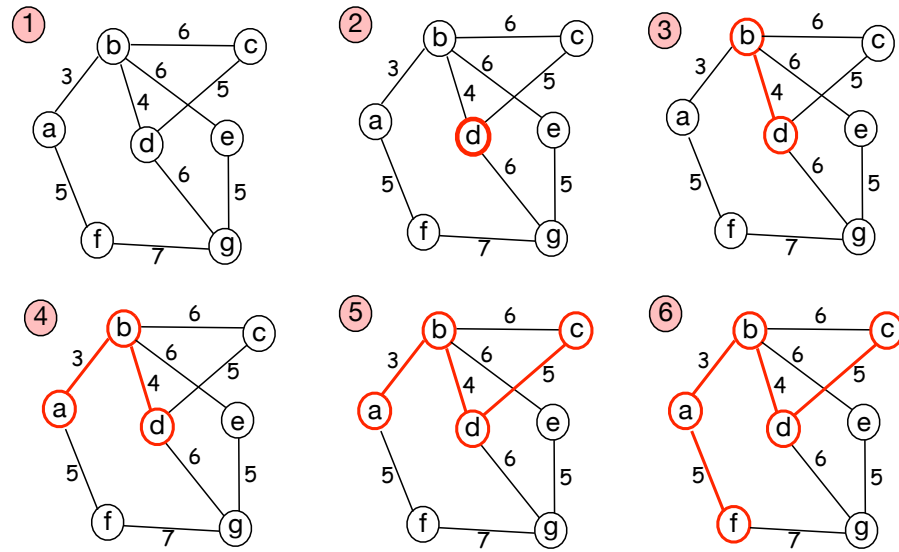


- First, it starts by selecting a random node (d) on the graph. Next, it starts to check all edges from the selected node and grows a tree by connecting to the node which,
(i) has the lowest weight, (ii) the connection does not form a cycle.
- It means the algorithm checks all neighbor nodes of the selected node and grows the tree by one edge. As a result, the randomly selected node of the previous step is connected to the node, which has the lowest cost.

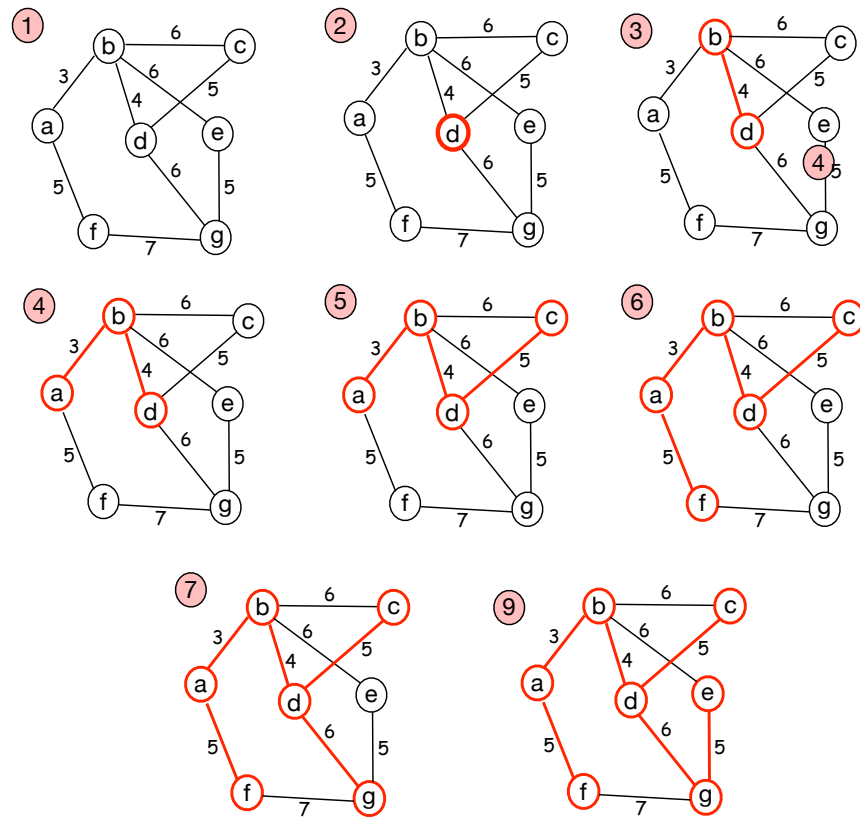
Prim Algorithm Example



Prim Algorithm Example



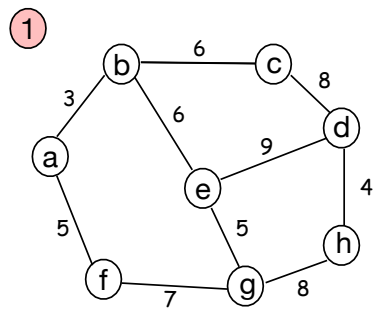
Prim Algorithm Example



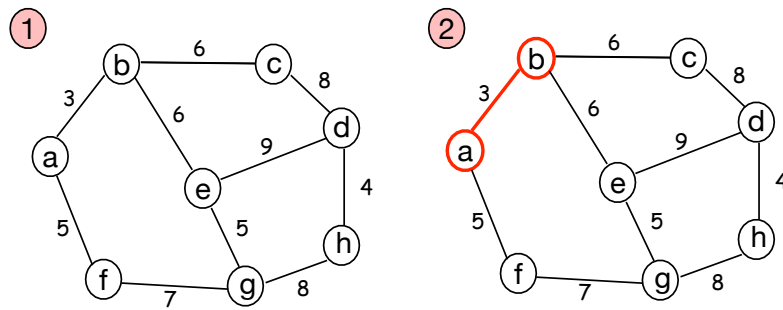
Kruskal Algorithm

- Another simple and popular MST algorithm is Kruskal algorithm, which has been introduced back in 1956.
- It starts by checking all edges in the graph and selects the smallest edge.
- Then by using these two nodes it constructs a single edge MST.
- Next, it repeats the process and check for the next smallest edge on the graph. It considers to add them into the MST, if the newly selected edge does not create a cycle with previously selected edges.
- This process continues until all nodes are added into the MST.

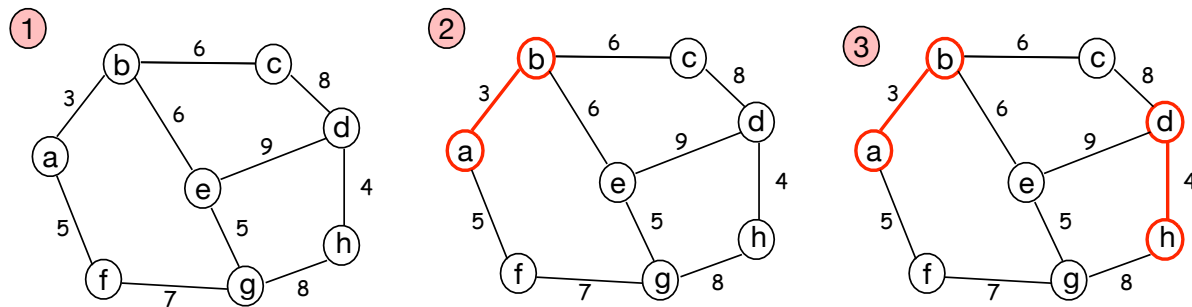
Kruskal Algorithm



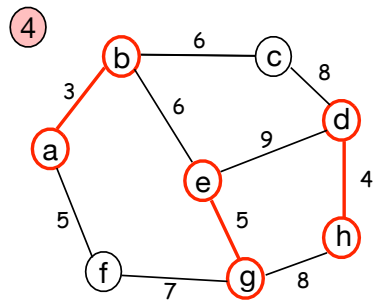
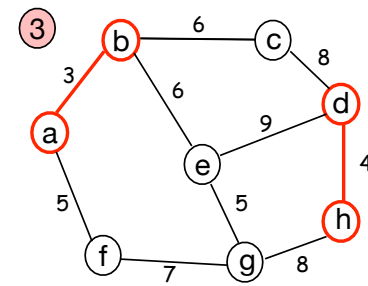
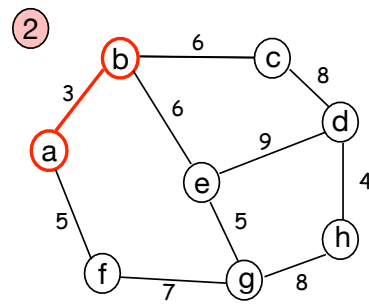
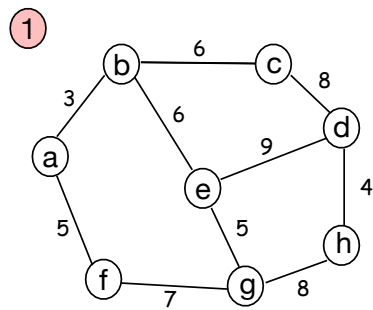
Kruskal Algorithm



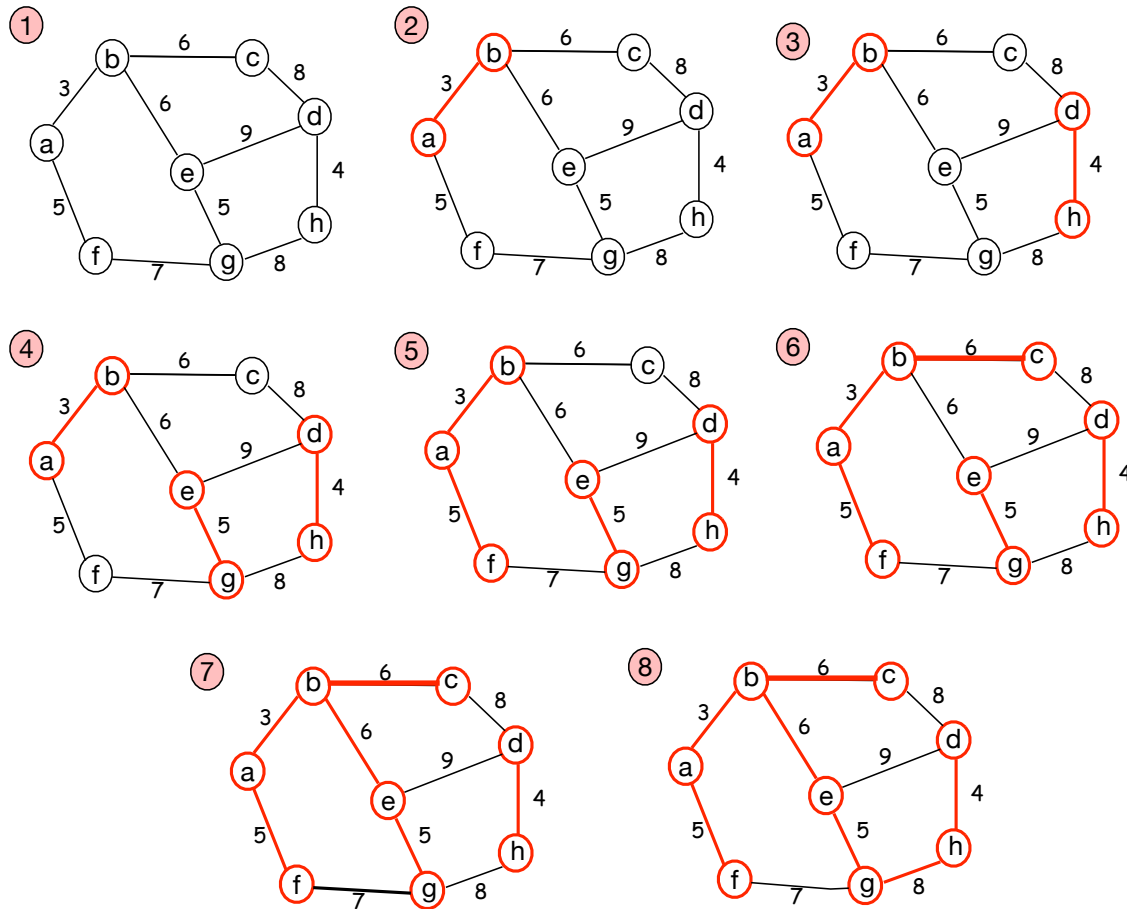
Kruskal Algorithm



Kruskal Algorithm



Kruskal Algorithm



Experiments in the Class

- <https://visualgo.net/en/mst>

Prim Algorithm Python Example

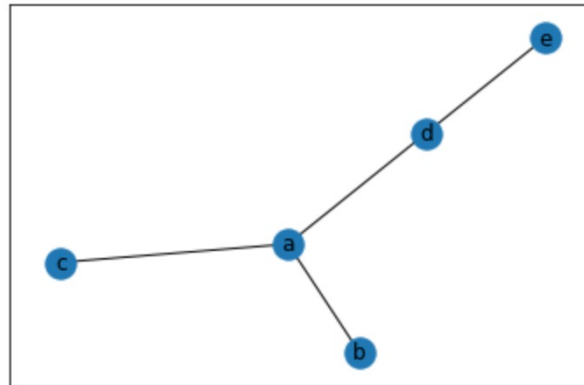
- [NetworkX](#) is a Python library for studying graphs and networks.
- Pip install networkx

```
In [1]: import networkx as nx  
import networkx.algorithms.tree as tree  
import matplotlib
```


Prim Algorithm Python Example

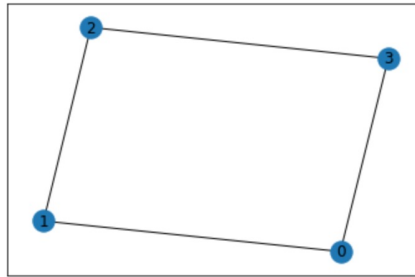
```
In [2]: G=nx.Graph()
# add nodes
G.add_node('a')
G.add_node('b')
G.add_node('c')
G.add_node('d')
G.add_node('e')
# add edges to the graph
G.add_edges_from([('a','c'),('b','a'),('a','d'),('d','e')])
print(G.number_of_nodes())
nx.draw_networkx(G)
```

5

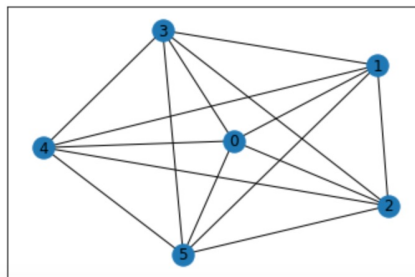


Prim Algorithm Python Example

```
In [3]: G2 = nx.cycle_graph(4)  
nx.draw_networkx(G2)
```

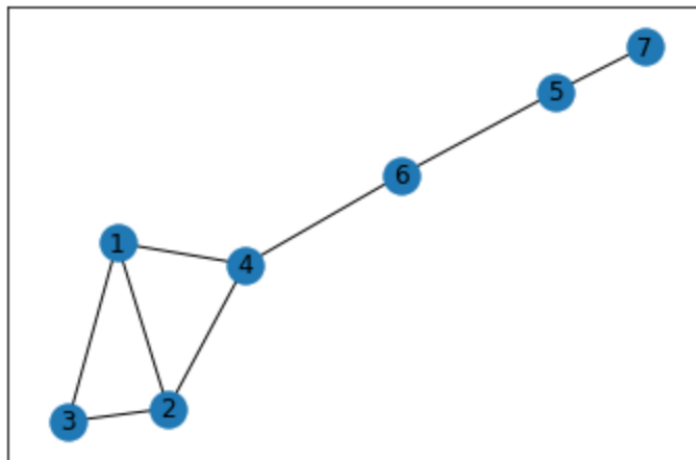


```
In [4]: G3 = nx.complete_graph(6)  
nx.draw_networkx(G3)
```



Prim Algorithm Python Example

```
In [5]: G4=nx.Graph()  
# form a weighted graph  
elist=[(1,2,1), (2,3,11), (1,3,7), (1,4,8), (4,2,12), (4,6,7), (5,6,6), (5,7,15)]  
G4.add_weighted_edges_from(elist)  
nx.draw_networkx(G4)
```

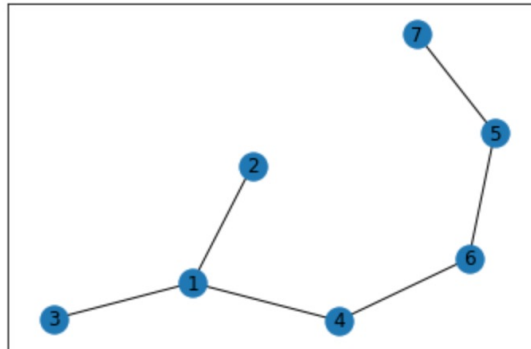


Prim Algorithm Python Example

```
In [6]: # create minimum spanning tree using prim algorithm
prim_mst=tree.minimum_spanning_edges(G4, algorithm="prim", data=False)
p_edgelist=list(prim_mst)
sorted(p_edgelist)
```

```
Out[6]: [(1, 2), (1, 3), (1, 4), (4, 6), (5, 7), (6, 5)]
```

```
In [7]: # plot minimum spanning tree using prim algorithm for G4
G5=nx.Graph()
G5.add_edges_from(p_edgelist)
nx.draw_networkx(G5)
```

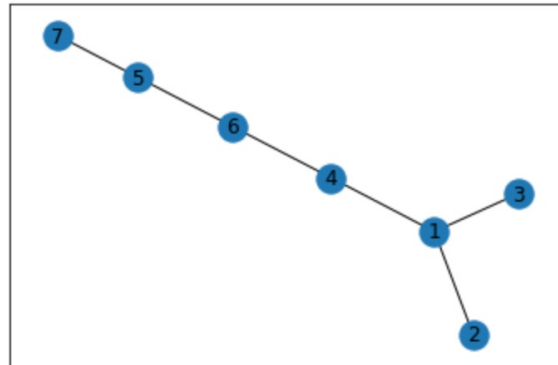


Kruskal Algorithm Python Example

```
In [8]: # create minimum spanning tree using kruskal algorithm
kruskal_mst=tree.minimum_spanning_edges(G4, algorithm="kruskal", data=False)
k_edgelist=list(kruskal_mst)
sorted(k_edgelist)
```

```
Out[8]: [(1, 2), (1, 3), (1, 4), (4, 6), (5, 7), (6, 5)]
```

```
In [9]: G5=nx.Graph()
G5.add_edges_from(k_edgelist)
nx.draw_networkx(G5)
```



Do we get the same minimum spanning trees with both Prim and Kruskal algorithms?

- In general:
 1. If the edge weights in the graph are all different from each other, then the graph has a unique minimum spanning tree. So, both Kruskal and Prim algorithms will return the same tree.
 2. If the edge weights in the graph are not all different then neither algorithm is necessarily deterministic.

Outline

- Basic Graph Concepts
- Minimum Spanning Tree
- **Shortest Path Finding** (Dijkstra, A*)
- Centrality Measurements
- Link Analysis
- Community Detection

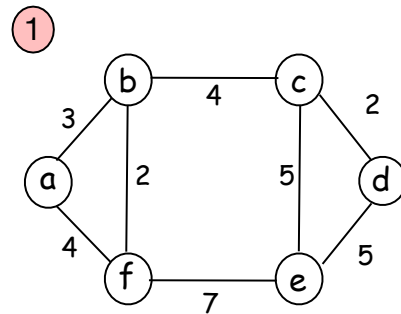
Dijkstra (di.ke.struh) Algorithm

Finding the shortest path

- It differs from the minimum spanning tree because the shortest distance between two vertices might not include all the vertices of the graph.
- A common application while working with graph is finding the shortest path. It is used in different applications including path finding in navigation systems, autonomous driving/flying, network routing, AI powered games, etc.
- Dijkstra algorithm (Dijkstra '59) uses the greedy method, greedy methods are heuristics algorithms that focus on making a locally optimum choice (immediate benefit) at each iteration.
- Dijkstra algorithm maintains the weight of edges that has been visited from the source node.
- The input of Dijkstra algorithm is a weighted graph. If the graph has direction, then the decision should be based on the direction, but for sake of brevity we use a graph which does not have a direction in the following example.

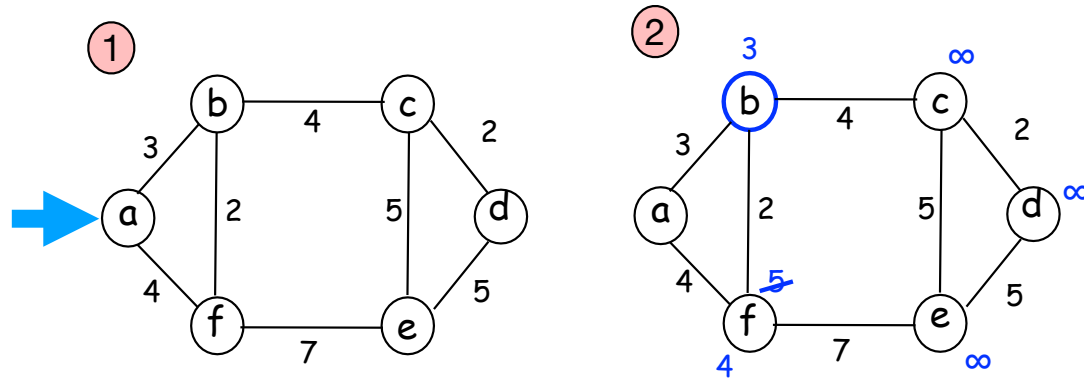
Dijkstra Algorithm

- The objective of the algorithm is to find the shortest path from vertex *a* to all other vertexes by using Dijkstra algorithm, the weight of each edges is written on the edge.



Dijkstra Algorithm

Starting from the **Figure 1**, the algorithm starts from node *a*. Node *a* has two adjacent nodes, *b* and *f*. The cost of reaching *b* and *f* nodes from node *a* is the weight of edges. The algorithm updates the cost of reaching them and this cost is written in blue color on top of each vertex in Figure. Note that nodes which are not adjacent to the current node (node *a*), receives the cost of infinity (∞) (see **Figure 2**). In other words, if there is no direct connection in between, the algorithm assigns ∞ as the cost.

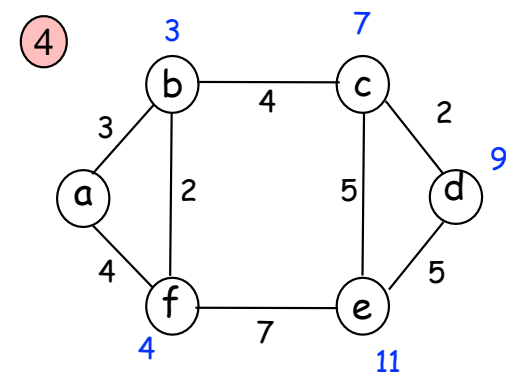
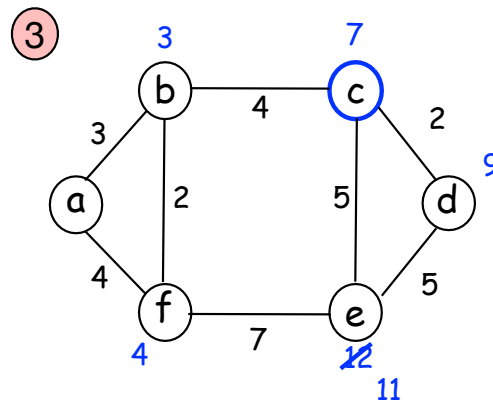
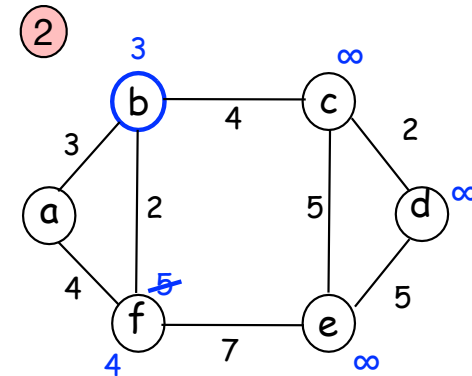
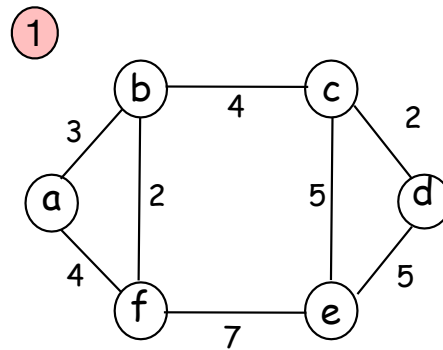


Dijkstra Algorithm

Next, node c is selected, because it has the shortest distance to b (after f , which the distance from a to f has already computed), as it is shown in Figure 2.

Then distances from a through c , distances to d and e , which are neighbors of c is computed, Figure 3.

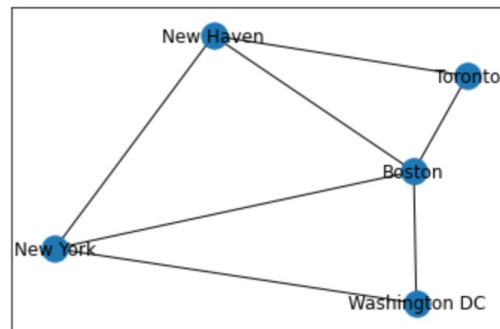
The distance from $a \rightarrow b \rightarrow c \rightarrow e$ is equal to 12, but there is a shorter distance available from f to e ($a \rightarrow f \rightarrow e$), which is 11, and thus the shortest path which is $a \rightarrow f \rightarrow e$ will be used as the shortest distance.



Dijkstra Algorithm Python Example

```
In [1]: import networkx as nx
import matplotlib.pyplot as plt
```

```
In [2]: G1=nx.Graph()
# form a weighted graph
elist=[("Boston","Washington DC",441), ("New York","Boston",215),
       ("Boston","New Haven",137),   ("New York","Washington DC",226),
       ("Toronto","Boston",885),     ("Toronto","New Haven",855),
       ("New York","New Haven",80) ]
G1.add_weighted_edges_from(elist)
nx.draw_networkx(G1)
```



```
In [3]: # apply dijkstra_path to find the shortest path between two nodes
print(nx.dijkstra_path(G1,"New Haven","Washington DC"))
```

```
['New Haven', 'New York', 'Washington DC']
```

Dijkstra Implementation

- <https://www.programiz.com/dsa/dijkstra-algorithm>
- https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.shortest_paths.weighted.dijkstra_path.html

References

- Müller, Andreas C.; Guido, Sarah. Introduction to Machine Learning with Python. O'Reilly Media. Kindle Edition.
- https://pyts.readthedocs.io/en/latest/auto_examples/approximation/plot_sax.html
- <https://pypi.org/project/instaloader/>
- Han, Jiawei, Pei, Jian, Tong, Hanghang. Data Mining (The Morgan Kaufmann Series in Data Management Systems). Elsevier Science. Kindle Edition.
- Hutter, Frank, Lars Kotthoff, Joaquin Vanschoren. Automated Machine Learning: Methods, Systems, Challenges (The Springer Series on Challenges in Machine Learning). Springer International Publishing. Kindle Edition.